Automated Accessibility Testing for Design Systems using AI

Goutham Ponnada
Software Engineer,
Accessibility Team

Ian McBurnie
Engineering Director,
Web Foundations Team

Wendy Reid
Senior Product Manager,
Accessibility Team

© 2026 eBay. All rights reserved. Confidential and proprietary.

For resources and slides:

https://is.gd/csun2026

Contents

01 Understanding your project

02 Tools and context

03 What we learned

04 Human in the loop

05 Conclusion

© 2026 eBay. All rights reserved. Confidential and proprietary.

Who is this for?

- Front End Developers
- Accessibility Subject Matter Experts
- Accessibility Engineers
- Product Managers
- QA Testers

AI presents huge risks and opportunities for accessible web development, what it takes is a **multi-disciplinary approach**.

Developers, SMEs, QAs, and PMs all have valuable information and context needed to build robust AI solutions for your teams.

© 2026 eBay. All rights reserved. Confidential and proprietary.

01

Understanding the current state of the project

© 2026 eBay. All rights reserved. Confidential and proprietary.

Evo Web

Evo Web

Components Accessibility Guides

Search ⌘ K

COMPONENTS

Accordion
Alert Dialog
Avatar
Badge
Breadcrumbs
Button
Calendar
Card
Carousel
Ccd
Chart Legend
Checkbox
Chip
Chips Combobox
Combobox
Confirm Dialog
Date Textbox
Details
Donut Chart
Education Notice
Eek
Field
File Input
File Preview Card Group
File Preview Card
Filter Chip
Filter Input

Evo Web > Components

Marko Components Storybook React Components Storybook

Accordion
A series of expanding/collapsing panels of content.
CSS v1.1 Marko v1.1 React v1.1

Alert Dialog
A lightbox dialog that prompts user for acknowledgement.
CSS v4.1 Marko v4.1 React v4.1

Avatar
A graphical representation of a user's identity.
CSS v1.0 Marko v1.0 React v1.0

Badge
A visual indicator that notifies users of numeric values.
CSS v1.2 Marko v1.2 React v1.2

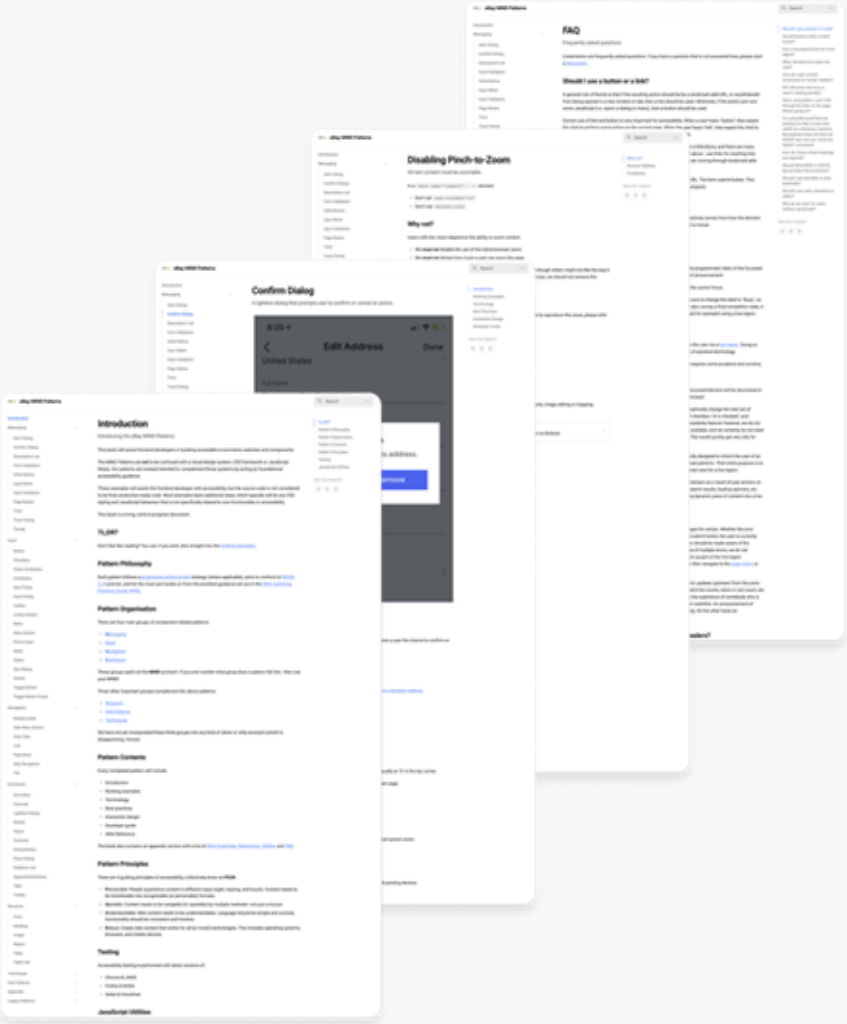
Breadcrumbs
A list of links representing current location in site hierarchy.
CSS v1.2 Marko v1.2 React v1.2

Button
A control for executing JavaScript or submitting/resetting form...
CSS v1.6 Marko v1.6 React v1.6

© 2026 eBay. All rights reserved. Confidential and proprietary.

Design-Agnostic Foundations (2015-2022)

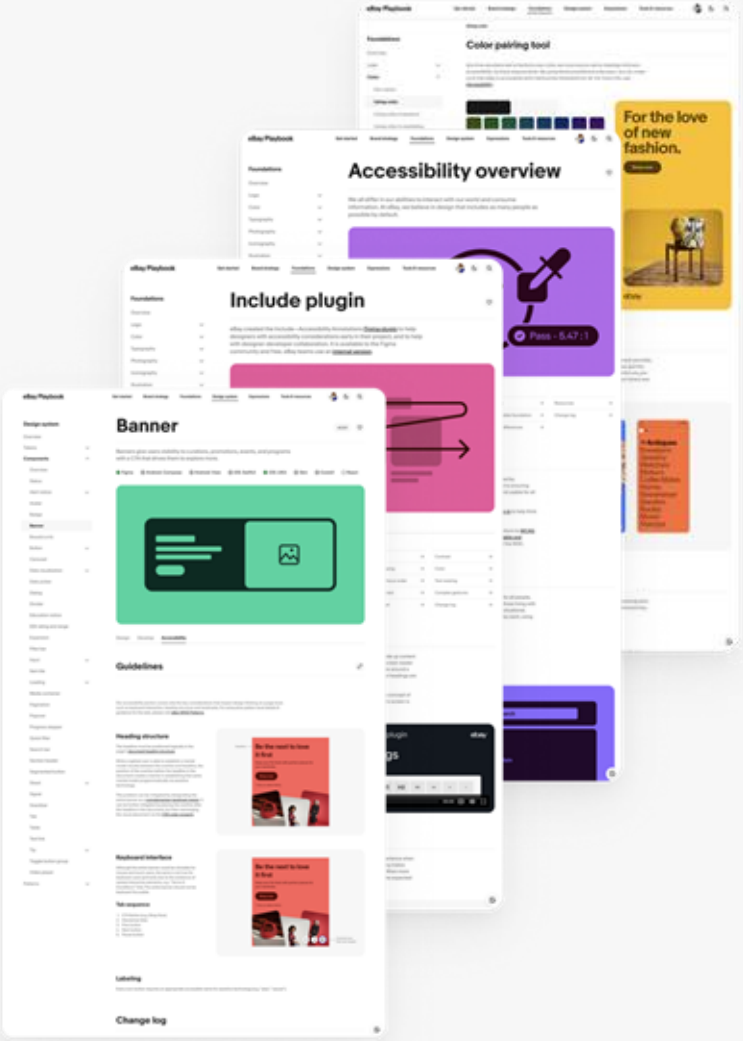
- MIND Patterns: interaction & accessibility guidance
- Design-agnostic pattern library
- Strong accessibility review
- Served as the foundation for future implementations



© 2026 eBay. All rights reserved. Confidential and proprietary.

Implementation Drift (2022-2025)

- Component implementations grew faster than foundations
- Marko components sometimes preceded pattern documentation
- Accessibility expectations not always encoded in tests
- Drift began to appear



© 2026 eBay. All rights reserved. Confidential and proprietary.

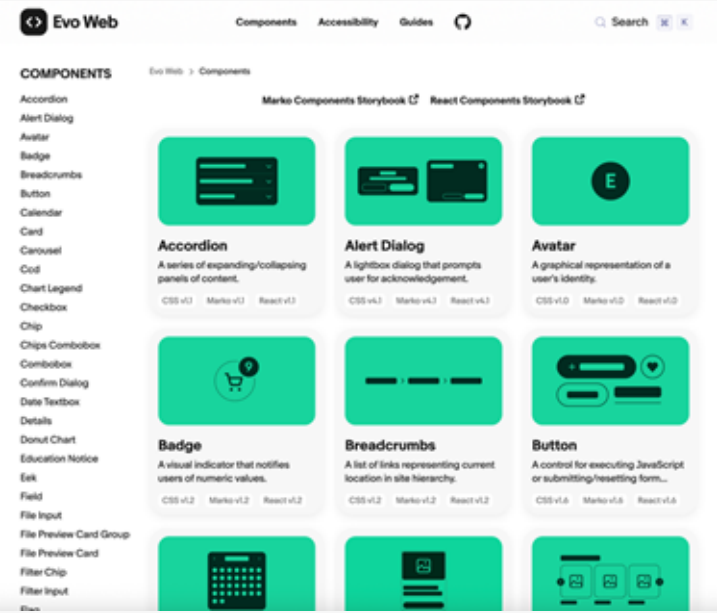
Guardrails Must Reinforce Themselves

Component-level enforcement surface

Explicit states & behaviors

Structured expectations

Automated validation



© 2026 eBay. All rights reserved. Confidential and proprietary.

Realizing we might have a gap

@accessibility - combobox issue. An external label is associated with the combobox using same **`for`** and **`id`** values, but JAWS is not announcing the label.

Could this be an issue with JAWS?

A team asked a question about accessibility support for a specific component, one that was correctly implemented but seemed to not be working.

Did we have a gap in component monitoring?

How can we monitor components over time?

© 2026 eBay. All rights reserved. Confidential and proprietary.

Project Structure and Tests

Two web component libraries: **ebayui-core** and **ebayui-core-react**.

Current State

80+ components:

- ➔ unit tests for:
  - ☑ rendering
  - — click events
  - ✕ keyboard interactions
  - ✕ ARIA usage
  - ✕ component states
- ➔ end to end tests for:
  - ✕ multi-step interactions
  - ✕ error handling
  - ✕ Interaction patterns
- ➔ automated checks
  - ☑ storybook accessibility tests

Desired State

80+ components:

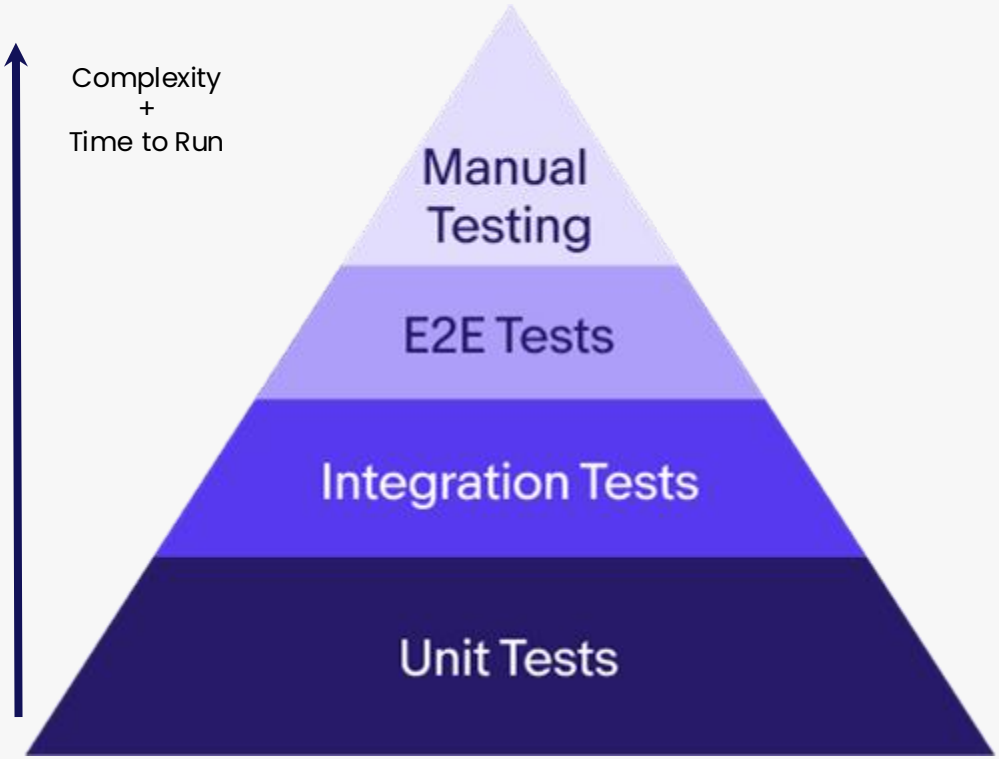
- ➔ unit tests for:
  - ☑ rendering
  - ☑ click events
  - ☑ keyboard interactions
  - ☑ ARIA usage
  - ☑ component states
- ➔ end to end tests for:
  - ☑ multi-step interactions
  - ☑ error handling
  - ☑ Interaction patterns
- ➔ automated checks
  - ☑ storybook accessibility tests

© 2026 eBay. All rights reserved. Confidential and proprietary.

Testing Basics: Test Types

No product should use one type of testing exclusively, you need a mix to ensure quality.

The foundation of a good test strategy is unit tests, followed by integration and end to end (E2E) tests, all of which can be automated. Manual testing completes the pyramid and ensures quality.



© 2026 eBay. All rights reserved. Confidential and proprietary.

Testing Basics: Unit and E2E Tests

Unit Tests

Most basic type of test, and should form the foundation of all software testing. There should be unit tests for each testable “unit” of code, such as a method, function, or class.

A unit test might look like this:

```
“when a button is clicked” {
    before {
        event.click(button);}
    “it emits click event” {
        expect(clickEvent) = 1;}
}
```

End to End (E2E) Tests

A more complex type of test, E2E tests focus on recreating expected user behaviours to ensure applications behave as expected. This is useful for testing multi-step flows or error states.

An E2E test might look like this:

```
“verify button loading state and label” {
    expect(button).toHaveLabel(“Loading”);
      load spinner = button.select(“.loading”);
    expect spinner.toBeVisible; }
      “verify button is interactive when loading” {
          focus.button(loading);
          expect(button).toHaveFocus;

}
```

© 2026 eBay. All rights reserved. Confidential and proprietary.

02

Tools and context engineering

© 2026 eBay. All rights reserved. Confidential and proprietary.

What we used

Evo web uses Storybook for UI development, testing, and documentation.

For this talk, we ran our prompts and context files to create tests using:

- Visual Studio Code (IDE)
- Cursor (IDE w/ built in AI Agents)
- Cline (AI Agent for VS Code)
- Claude Code (AI Agent run in Terminal or in VS Code)
- Copilot (AI Agent for VS Code)
- Claude Sonnet 4.5 (AI model)
- Claude Opus 4.6 (AI Model)
- Cursor Composer 1.5 (AI model)
- Chat GPT 5.2 (AI Model)

© 2026 eBay. All rights reserved. Confidential and proprietary.

Documentation and Context Engineering

Context Engineering

"Context engineering involves curating, integrating, and orchestrating diverse data sources, memory mechanisms, and environmental signals so that AI systems have the relevant background needed to perform reliably and accurately on complex tasks." - Intuition Labs.

Context files are the cheat sheet for completing the task. They contain essential information the tool needs to find information, orient itself in the project, and complete the tasks it is given.

Context Windows

Every AI tool on the market has a "context window", a limited space for information. Different models have different sized windows, but the common range is 200k-1 million "tokens" (approx. 150k-750k words). The context window includes both your inputs and the outputs of the tool.

Tools can only look at and reference content within the context window.

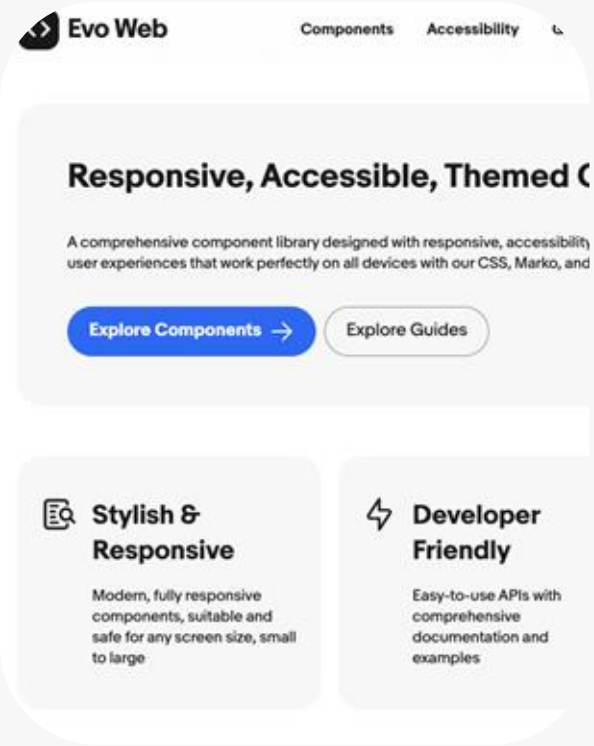
Documentation

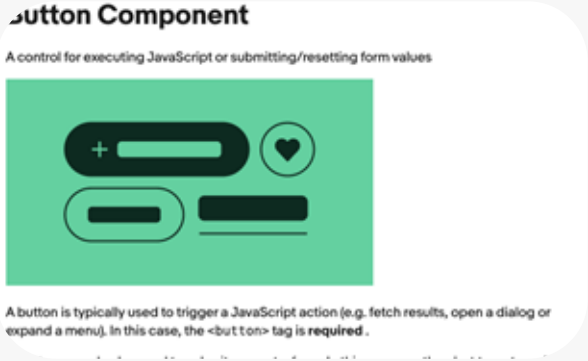
Documentation can be in any* format, but it needs to be accessible to the tools. These can be local files, hosted online, or publicly available materials.

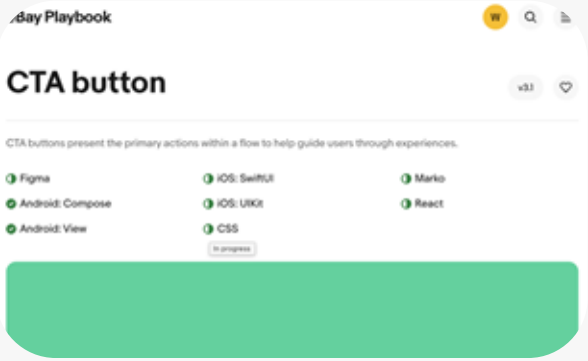
** well-structured or simple document formats generally perform better, such as HTML, MarkDown, XML, JSON, and CSV.*

© 2026 eBay. All rights reserved. Confidential and proprietary.

Documentation Sources







© 2026 eBay. All rights reserved. Confidential and proprietary.

03

What we learned

© 2026 eBay. All rights reserved. Confidential and proprietary.

Learning #1

You will save time in the long run, but there will be a lot of work along the way.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Make the tools work for you

Use “plan” actions to prepare, “act” once you are happy with the shape of the plan.

Iterate on your context files and prompts and encourage AI tools to clarify and question things along the way.

Every mistake is an opportunity for improvement.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Learning #2

AI coding agents love to make assumptions.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Common assumptions

Naming/Locating

Assuming every single component had a “default” and setting locators for all tests to the default page.

Browser behaviours

Assuming all tests could use the same keyboard shortcuts or would follow the same pattern.

Handling “args”

Storybook uses “args” to define how a component should render, but not all of our components used args in the same way. The tools assumed they did.

Usage of ARIA

Assuming which ARIA attributes should be used, writing tests for those attributes, and not checking whether they were present or not.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Learning #3

Humans do too.

© 2026 eBay. All rights reserved. Confidential and proprietary.

AI tools can reveal the weaknesses in your own work.

Project consistency

Why didn’t all our component have a “default” state? Why weren’t we using args consistently?

Assuming clarity

Our combobox component is a combination of listbox and textbox, but what does that actually mean?

Missing documentation

Several components didn’t have overview or accessibility documentation to refer to.

Assuming quality

Many components had existing tests, but were they robust enough?

© 2026 eBay. All rights reserved. Confidential and proprietary.

Result of assumptions

Hallucinated Interactions

Without explicit "only test what's documented" constraints, the model will invent keyboard interactions that don't exist on the component.

Example: it'll generate Arrow key navigation tests for ebay-menu-button because buttons in other design systems support arrow key groups — but eBay's menu button may not.

"Comprehensive" Means Everything

Without a scope constraint, the model assumes comprehensive = exhaustive. For a component like ebay-menu, it'll try to generate tests for every possible keyboard combo, every ARIA attribute on every element, every state permutation. You'll get 40+ test cases when you probably wanted 10-15 focused ones covering the documented behavior. Then someone has to triage which ones are real and which are noise.

© 2026 eBay. All rights reserved. Confidential and proprietary.

How we adapted as we learned

As we noticed patterns emerging, we addressed them through updates to the context file.

Sometimes the updates helped, in other cases they did not! Finding the right way to describe or reference things is an art that takes time to develop.

For example, in response to an issue where it kept failing to find the right web pages for a component, we needed to add:

Note: Component names in URLs don't include the `ebay-` prefix (e.g., `ebay-button` → `/components/button`)

© 2026 eBay. All rights reserved. Confidential and proprietary.

04

Why we keep the human in the loop

© 2026 eBay. All rights reserved. Confidential and proprietary.

Learning #4

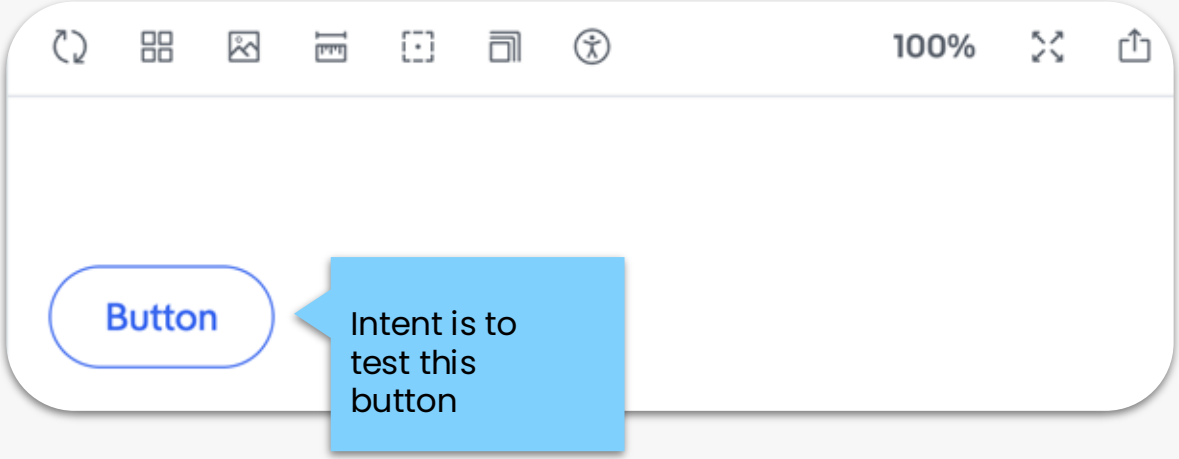
Prevent automation bias & false security

© 2026 eBay. All rights reserved. Confidential and proprietary.

Tests passed, auditing the wrong button

```
describe("given a standard button", () => {

    it("then it has correct role", () => {
        const button = component.getByRole("button");
        expect(button).toBeTruthy();
    });
```



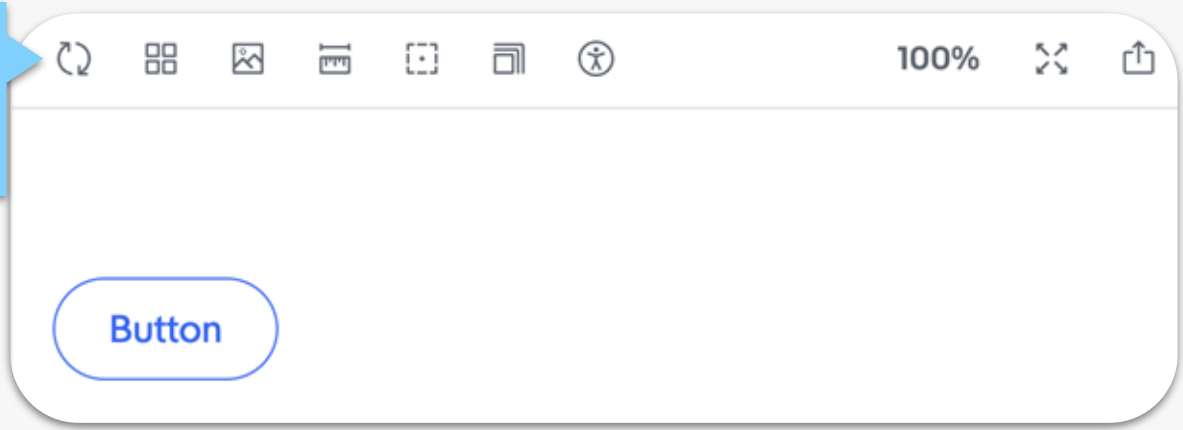
© 2026 eBay. All rights reserved. Confidential and proprietary.

Tests passed, auditing the wrong button

```
describe("given a standard button", () => {

    it("then it has correct role", () => {
        const button = component.getByRole("button");
        expect(button).toBeTruthy();
    });
```

Instead, scoped to tested this



© 2026 eBay. All rights reserved. Confidential and proprietary.

Tests tripled, but the value dropped to zero

These tests provide no value

```
describe("given button with different sizes", () => {
    it("then large button maintains accessibility", async () => {
        component = await render(template, {
            renderBody: "Large",
            size: "large",
        });
        const button = component.getByRole("button");
        expect(button.textContent.trim()).toBe("Large");
    });

    it("then small button maintains accessibility", async () => {
        component = await render(template, {
            renderBody: "Small",
            size: "small",
        });
        const button = component.getByRole("button");
        expect(button.textContent.trim()).toBe("Small");
    });
});
```

© 2026 eBay. All rights reserved. Confidential and proprietary.

Human review ensures tests align with reality

What we observed?

- **Models hallucinate:** AI assumes behavior and misrepresents intent
- **Accessibility isn't guaranteed:** AI need human defined context
- **Prompting requires guardrails:** Stop duplication and test bloat.

© 2026 eBay. All rights reserved. Confidential and proprietary.

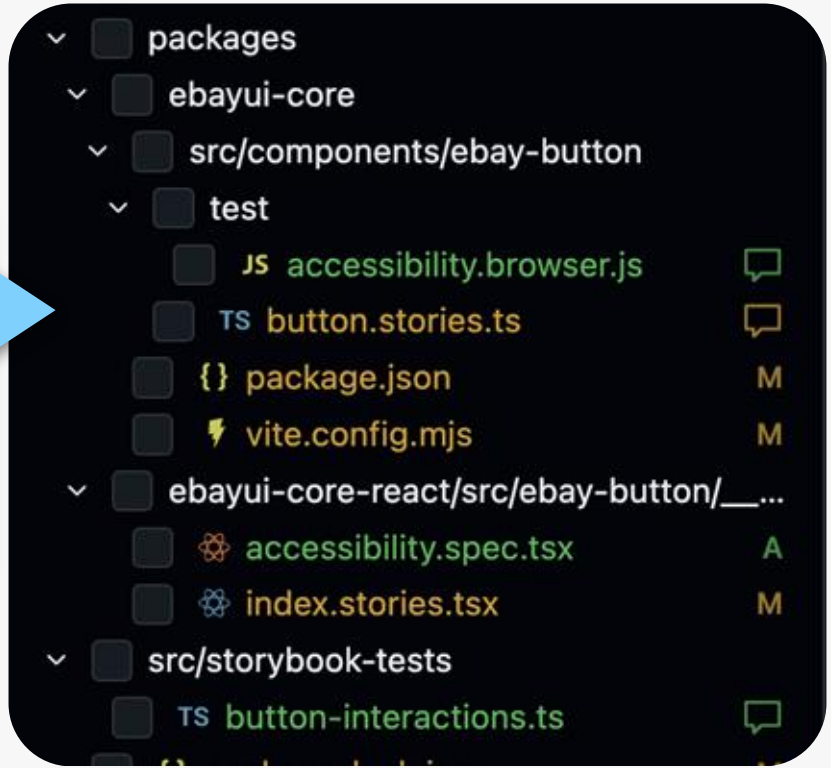
Learning #5

The "Moving Bottleneck"

© 2026 eBay. All rights reserved. Confidential and proprietary.

Review, debug & verify

Number of files changed with single prompt when asked to add tests for button



© 2026 eBay. All rights reserved. Confidential and proprietary.

Prioritize human review in the development lifecycle

What we observed?

- **AI as Junior Dev**: Rapid code generation is only as fast as our ability to verify it.
- **The Accumulation Risk:** We aren't just shipping features; we're stockpiling unverified code.
- **Shifting from doing to critiquing** creates a hidden cost.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Learning #6

Fighting the Cognitive Debt

© 2026 eBay. All rights reserved. Confidential and proprietary.

If the bottleneck has shifted to review, why pile more work into planning?

What we observed?

- **Writing vs. Approving:** We are trading the creation of tests for the curation of code.
- **The Knowledge Gap:** We are losing the "Deep Understanding" of our own test suites.
- **The Shift:** We must move verification from Review back to Planning.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Learning #7

Your site can meet compliance and still remain a nightmare to navigate.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Balance Compliance with Empathy

```
describe("given any button variant", () => {
    beforeEach(async () => {…
    });

    it("then it has sufficient text content or accessible name", () => {
        const button = component.getByRole("button");
        const text = button.textContent.trim();
        const ariaLabel = button.getAttribute("aria-label");

        expect(text || ariaLabel).toBeTruthy();
    });

    it("then it has minimum dimensions for touch target", () => {
        const button = component.getByRole("button");
        const rect = button.getBoundingClientRect();

        expect(rect.width).toBeGreaterThan(0);
        expect(rect.height).toBeGreaterThan(0);
    });
});
```

Bad at judging whether a label is actually meaningful

© 2026 eBay. All rights reserved. Confidential and proprietary.

Ensure the "Spirit" is met

What we observed?

- **AI enforces the "Letter":** Does this meet WCAG 2.2?
- **Humans feels the "Spirit":** Is this component frustrating, confusing, or broken?
- Compliance is a floor, not a ceiling

© 2026 eBay. All rights reserved. Confidential and proprietary.

Learning #8

Every gap in your prompt is filled by the model's assumptions.

© 2026 eBay. All rights reserved. Confidential and proprietary.

Fill the gaps

Adopt a Persona

We started by defining the "Who." Before the AI processes anything, it needs the context of an expert's lens (e.g., "Act as a QE Accessibility Lead").

Collaborate with SME

AI needs expert opinion to get accessibility right.

Instead of watching from the sidelines, we have a unique opportunity to teach these systems our values.

Give model “Time to Think”

Explicitly asking the AI to think step-by-step. Nudging it away from guessing.

We found that the most successful strategy wasn't just asking the AI for code, but forcing it into a **Plan** → **Act** → **Reflect** cycle.

Refine your prompts

The less robust the model is, the more fiddling is needed.

Provide sufficient context neither less nor more.

© 2026 eBay. All rights reserved. Confidential and proprietary.

05

Conclusion

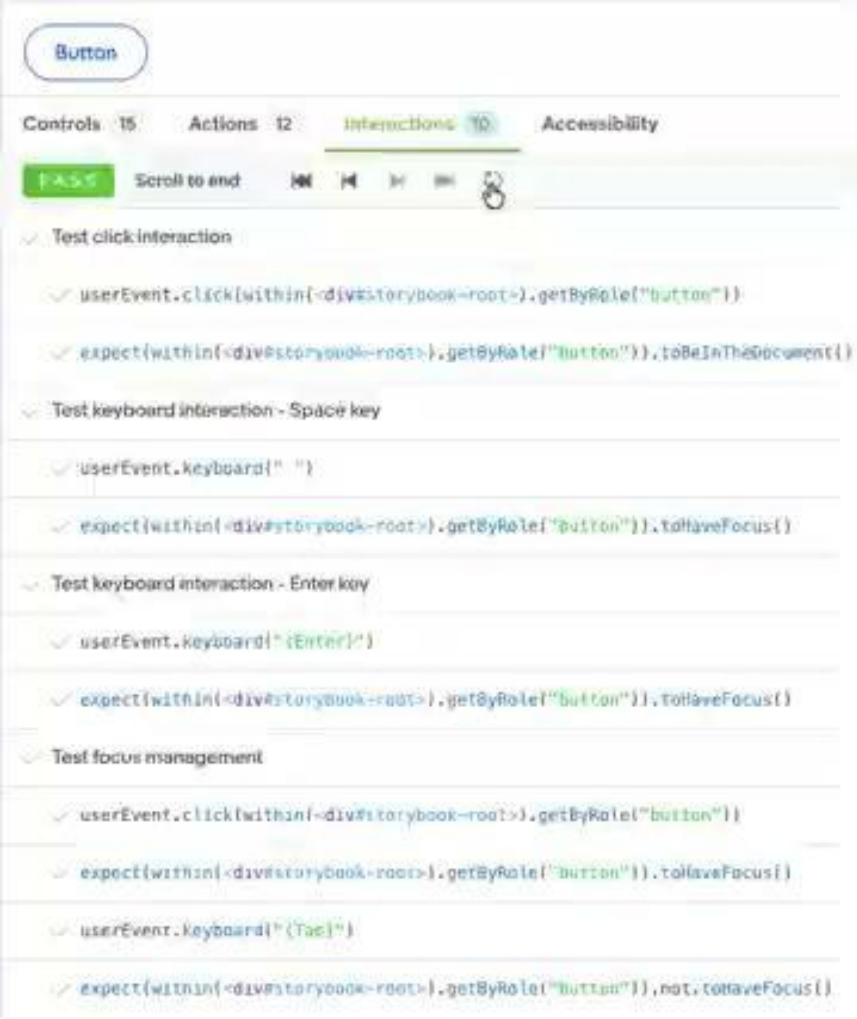
© 2026 eBay. All rights reserved. Confidential and proprietary.

What we accomplished

68 unit tests for the Marko and React ebay-button components.

(y) interaction tests in Storybook.

Context files and prompt templates that can be used to write tests for the remaining components.



© 2026 eBay. All rights reserved. Confidential and proprietary.

AI is just another tool to do our work.

Ignore the hype, focus on what makes the most sense for your team, your skills, and your technology.

These tools are not magic, but they can change the type of work you do.

© 2026 eBay. All rights reserved. Confidential and proprietary.

It takes a village

Reaching this point required a team with multiple skill sets and a foundation of knowledge built by many.

No single person or technology could have done this. Leverage the skills and knowledge you and your teams have.

© 2026 eBay. All rights reserved. Confidential and proprietary.

References

Slides: https://is.gd/csun2026

- Evo Web
- Accessibility LLM Evaluation Report
- AI Model Accessibility Checker
- Context Engineering
- Three Paradoxes of AI-assisted Engineering
- How Generative and Agentic AI Shift Concern from Technical Debt to Cognitive Debt
- AI Engineering by Chip Huyen

© 2026 eBay. All rights reserved. Confidential and proprietary.